TECHNICAL UNIVERSITY OF MOLDOVA
Faculty of Computers, Informatics, and Microelectronics
Department of Software Engineering
Software Engineering Study Programme

REPORT ON LABORATORY WORK NO. 6

OPERATING SYSTEMS

Inside the xv6 File System

Done by: st. gr. FAF-241, Lucian-Adrian Gavril

Checked by: univ. assist., Reițman Patricia

Chișinău, 2026

TABLE OF CONTENTS

INTRODUCTION 3

1 DIRECTORIES AND FILE TYPES 4

1.1 Types, the root inode and the directory entry 4

1.2 ls in xv6 4

2 MYSTAT AND THE LIFE OF A LINK 6

2.1 user/mystat.c 6

2.2 Links and unlinking 7

2.3 Where the blocks are really freed 7

3 LIMITS AND DISK LAYOUT 9

3.1 The constants and the on-disk inode 9

3.2 Names longer than 14 characters 9

3.3 The disk layout 10

4 USER/FIND.C 11

4.1 How it works 11

4.2 Testing find 13

CONCLUSIONS 15

REFERENCES 16

INTRODUCTION

Purpose of the work. To repeat the Lab 5 experiments inside xv6, where the whole file system can be read in the source. The work covers how directories store names, what an inode holds, how links and unlinking work down to the line where blocks are freed, which hard limits follow from the constants in `kernel/fs.h`, and two new user programs: `mystat`, which prints an inode, and `find`, which walks a directory tree recursively.

Environment. Debian GNU/Linux 13, `riscv64-linux-gnu` GCC 14, QEMU 10.0. xv6 is the current upstream `mit-pdos/xv6-riscv` with `user/mystat.c` and `user/find.c` added to `UPROGS` (together with `sleep` and `args` from Lab 4). Each QEMU session started from a freshly built `fs.img`. Every terminal figure is the real output of the commands shown in it.

1 DIRECTORIES AND FILE TYPES

1.1 Types, the root inode and the directory entry

xv6 knows three file types, defined in `kernel/stat.h`: `T_DIR` = 1, `T_FILE` = 2, `T_DEVICE` = 3. The root directory is inode `ROOTINO` = 1. A directory is a regular array of `struct dirent`: a 2-byte inode number `inum` and a 14-byte name `name[DIRSIZ]`, 16 bytes per entry (Figure 1.1).

```
dev1@dev1:~/xv6-riscv$ grep -n 'T_DIR\|T_FILE\|T_DEVICE' kernel/stat.h
1:#define T_DIR    1 // Directory
2:#define T_FILE   2 // File
3:#define T_DEVICE 3 // Device
dev1@dev1:~/xv6-riscv$ grep -n 'ROOTINO\|define DIRSIZ' kernel/fs.h
4:#define ROOTINO 1    // root i-number
54:#define DIRSIZ 14
dev1@dev1:~/xv6-riscv$ grep -n -A3 '^struct dirent' kernel/fs.h
58:struct dirent {
59-  ushort inum;
60-  char name[DIRSIZ] __attribute__((nonstring));
61-};
```

Figure 1.1 – File types, ROOTINO and struct dirent in the source

1.2 ls in xv6

xv6 `ls` prints name, type, inode and size for each entry (Figure 1.2). In `/`, both `.` and `..` are inode 1: the root is its own parent. The programs are type 2 with inodes 3, 4, …, assigned in `UPROGS` order by `mkfs`, and `console` is type 3, a device with size 0. A new directory `a` has size 32: exactly two entries, `.` (its own inode) and `..` (inode 1), $2 \times 16 = 32$ bytes.

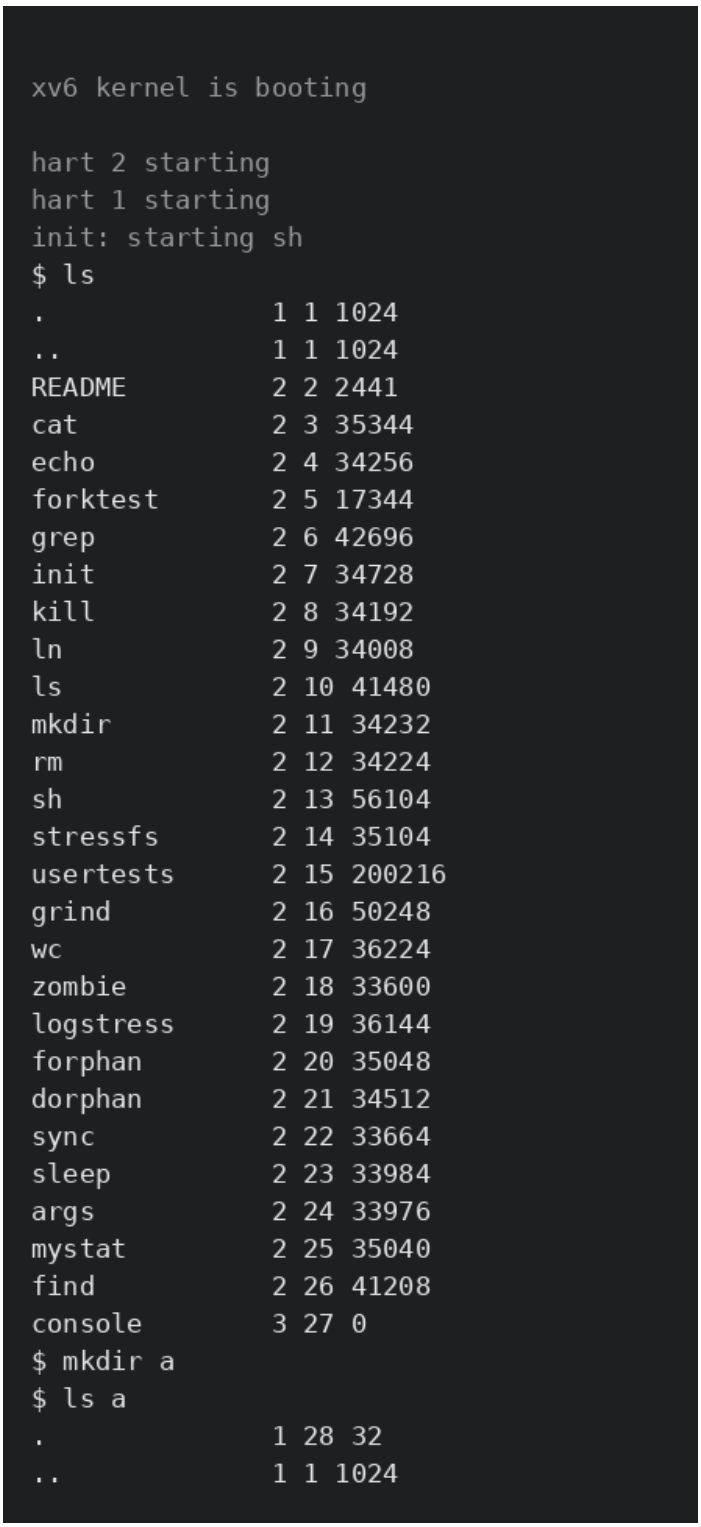


Figure 1.2 – ls of the root directory and of a new directory

2 MYSTAT AND THE LIFE OF A LINK

2.1 user/mystat.c

`mystat` calls `stat()` for every argument and prints the type as a word, the inode number, the link count and the size. A path that cannot be resolved is reported on stderr, and the program continues with the remaining arguments (Figure 2.1).

```
dev1@dev1:~/xv6-riscv$ cat user/mystat.c
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

char *
typename(int t)
{
  switch(t) {
  case T_DIR:
    return "directory";
  case T_FILE:
    return "file";
  case T_DEVICE:
    return "device";
  default:
    return "?";
  }
}

int
main(int argc, char *argv[])
{
  struct stat st;

  if(argc < 2){
    fprintf(2, "usage: mystat <path...>\n");
    exit(1);
  }

  for(int i = 1; i < argc; i++){
    if(stat(argv[i], &st) < 0){
      fprintf(2, "mystat: cannot stat %s\n", argv[i]);
      continue;
    }
    printf("%s: %s, inode %d, links %d, size %ld bytes\n",
           argv[i], typename(st.type), st.ino, st.nlink, st.size);
  }
  exit(0);
}
```

Figure 2.1 – The source of user/mystat.c

2.2 Links and unlinking

`ln README r2` creates a second directory entry for inode 2: both names show `links 2` and the same size, 2441. `rm README` removes only the entry. The count falls to 1, and `r2` still holds the full text: `cat r2 | wc` prints `48 336 2441` (Figure 2.2).

Two details differ from Linux. The root `.` shows `links 2` and the new directory `a` shows `links 1`. xv6 does not count a directory's own `.` entry, so a new directory has one link, its name in the parent. Every subdirectory then adds one to its parent through `..`, which is why `/` went from 1 to 2 when `a` was created.

```
$ mystat README . console a
README: file, inode 2, links 1, size 2441 bytes
.: directory, inode 1, links 2, size 1024 bytes
console: device, inode 27, links 1, size 0 bytes
a: directory, inode 28, links 1, size 32 bytes
$ ln README r2
$ mystat README r2
README: file, inode 2, links 2, size 2441 bytes
r2: file, inode 2, links 2, size 2441 bytes
$ rm README
$ mystat r2
r2: file, inode 2, links 1, size 2441 bytes
$ cat r2 | wc
48 336 2441
$ mystat nosuch
mystat: cannot stat nosuch
```

Figure 2.2 – mystat, a hard link and unlink

2.3 Where the blocks are really freed

`sys_unlink()` in `kernel/sysfile.c` clears the directory entry and does `ip->nlink--` (line 244). It does not free any data. Line 239 is the extra decrement for removing a directory (its `..` pointed to the parent), and line 178 rolls back a failed `link()`. Data is freed only in `iput()` in `kernel/fs.c`. When the last in-memory reference is dropped (`ref == 1`) and the on-disk count is 0 (`nlink == 0`), `itrunc()` returns every direct and indirect block to the free bitmap (lines 357–366, Figure 2.3). So a file that is deleted while a process still has it open stays readable until the process closes it, which is the same rule observed on ext4 in Lab 5.

```
dev1@dev1:~/xv6-riscv$ grep -n 'nlink--' kernel/sysfile.c
178:  ip->nlink--;
239:    dp->nlink--;
244:  ip->nlink--;
dev1@dev1:~/xv6-riscv$ grep -n -B2 -A10 'ip->ref == 1 && ip->valid && ip->nlink == 0' kernel/fs.c
355-  // since once ref hits 0, ip may be recycled by a concurrent iget()
356-  // for a different inum.
357:  int last = (ip->ref == 1 && ip->valid && ip->nlink == 0);
358-  uint dev = ip->dev, inum = ip->inum;
359-
360-  if (last) {
361-    // ip->ref == 1 means no other process can have ip locked.
362-    acquiresleep(&ip->lock);
363-    release(&itable.lock);
364-
365-    itrunc(ip); // free the data blocks (type stays nonzero on disk)
366-    ip->valid = 0;
367-
```

Figure 2.3 – nlink– in sysfile.c and the freeing condition in iput()

3 LIMITS AND DISK LAYOUT

3.1 The constants and the on-disk inode

```
dev1@dev1:~/xv6-riscv$ grep -n 'define BSIZE\|define NDIRECT\|define NINDIRECT\|define MAXFILE\|define D
IRSIZ' kernel/fs.h
5:#define BSIZE   1024 // block size
26:#define NDIRECT   12
27:#define NINDIRECT (BSIZE / sizeof(uint))
28:#define MAXFILE   (NDIRECT + NINDIRECT)
54:#define DIRSIZ 14
dev1@dev1:~/xv6-riscv$ grep -n 'define FSSIZE\|define LOGBLOCKS\|define NINODES' kernel/param.h mkfs/mkf
s.c
kernel/param.h:10:#define LOGBLOCKS   (MAXOPBLOCKS * 3) // max data blocks in on-disk log
kernel/param.h:12:#define FSSIZE      2000              // size of file system in blocks
mkfs/mkfs.c:23:#define NINODES 200
dev1@dev1:~/xv6-riscv$ grep -n -A7 '^struct dinode' kernel/fs.h
32:struct dinode {
33-  short type;              // File type
34-  short major;             // Major device number (T_DEVICE only)
35-  short minor;             // Minor device number (T_DEVICE only)
36-  short nlink;             // Number of links to inode in file system
37-  uint size;               // Size of file (bytes)
38-  uint addrs[NDIRECT + 1]; // Data block addresses
39-};
```

Figure 3.1 – Constants in fs.h and param.h, and struct dinode

The on-disk inode `struct dinode` has only `type`, `major`, `minor`, `nlink`, `size` and 13 block addresses: $4 \times 2 + 4 + 13 \times 4 = 64$ bytes. Compared with ext4 it lacks owner, permission bits, timestamps and symbolic links, so xv6 has no `chmod`, no users and no `ls -l` times.

Limit	From	Value
Block size	`BSIZE`	1024 bytes
Direct blocks	`NDIRECT`	12 → 12 288 bytes
Pointers in the indirect block	`NINDIRECT` = 1024 / 4	256 → 262 144 bytes
Maximum file size	`MAXFILE` = 12 + 256	268 blocks = **274 432 bytes (268 KiB)**
Maximum name length	`DIRSIZ`	14 characters
Number of inodes	`NINODES`	200
File system size	`FSSIZE`	2000 blocks = 2000 KiB

Table 3.1 – Limits that follow from the constants

3.2 Names longer than 14 characters

`mkdir averyveryverylongname` succeeds, but `ls` shows `averyveryveryl`, the first 14 characters (Figure 3.2). The same happens with files: `echo hi > abcdefghijklmnopq` creates an entry named `abcdefghijklmn`, and `cat abcdefghijklmn` prints `hi`. The long name itself also keeps working (`mystat abcdefghijklmnopq` finds inode 30). `namecmp()` compares only `DIRSIZ` bytes, so every name that shares

the first 14 characters refers to the same file. Linux would reject such a name with `ENAMETOOLONG` instead of truncating it silently.

```
$ mkdir averyveryverylongname
$ ls | grep avery
averyveryveryl 1 29 32
$ echo hi > abcdefghijklmnopq
$ cat abcdefghijklmn
hi
$ mystat abcdefghijklmnopq
abcdefghijklmnopq: file, inode 30, links 1, size 3 bytes
```

Figure 3.2 – Names truncated to 14 characters

3.3 The disk layout

`mkfs` prints the layout of the 2000-block image (Figure 3.3):

Blocks	Count	Content
0	1	boot block (unused by xv6)
1	1	superblock
2–32	31	log (1 header + 30 data blocks, `LOGBLOCKS`)
33–45	13	inodes: 200 inodes $\times$ 64 B = 12 800 B, at 16 per block $\rightarrow \lceil 200/16 \rceil = 13$
46	1	free-block bitmap (2000 bits fit in one 8192-bit block)
47–1999	1953	data blocks

Table 3.2 – xv6 disk layout

`balloc: write bitmap block at sector 46` confirms the position of the bitmap. After copying `README` and all user programs, 1101 blocks were already in use.

```
dev1@dev1:~/xv6-riscv$ rm -f fs.img ; make fs.img 2>&1 | grep -E 'nmeta|balloc'
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953 total 2000
balloc: first 1101 blocks have been allocated
balloc: write bitmap block at sector 46
```

Figure 3.3 – The mkfs summary of fs.img

4 USER/FIND.C

4.1 How it works

`find <dir> <name> [-n]` opens the directory and checks with `fstat()` that it really is one. It then reads it one `struct dirent` at a time and skips free slots (`inum == 0`). The 14-byte name is copied into a buffer one byte longer so that it can be terminated even when it is exactly 14 characters long. It skips `.` and `..`; otherwise the recursion would loop forever, going down into itself or back up into its parent. For each entry it builds `path/name`, prints it if the name matches (with the inode number when `-n` is given), and recurses into subdirectories. Every opened directory is closed before returning, so deep trees do not exhaust the 16 file descriptors a process has (`NOFILE`). It also refuses paths that would overflow its 512-byte buffer (Figure 4.1).

```
dev1@dev1:~/xv6-riscv$ sed -n '9,75p' user/find.c
void
find(char *path, char *target)
{
  char buf[512], *p;
  int fd;
  struct dirent de;
  struct stat st;

  if((fd = open(path, O_RDONLY)) < 0){
    fprintf(2, "find: cannot open %s\n", path);
    return;
  }

  if(fstat(fd, &st) < 0){
    fprintf(2, "find: cannot stat %s\n", path);
    close(fd);
    return;
  }

  if(st.type != T_DIR){
    close(fd);
    return;
  }

  if(strlen(path) + 1 + DIRSIZ + 1 > sizeof(buf)){
    fprintf(2, "find: path too long\n");
    close(fd);
    return;
  }

  strcpy(buf, path);
  p = buf + strlen(buf);
  *p++ = '/';

  while(read(fd, &de, sizeof(de)) == sizeof(de)){
    char name[DIRSIZ + 1];

    if(de.inum == 0)
      continue;

    memmove(name, de.name, DIRSIZ);
    name[DIRSIZ] = 0;

    if(strcmp(name, ".") == 0 || strcmp(name, "..") == 0)
      continue;

    strcpy(p, name);

    if(stat(buf, &st) < 0){
      fprintf(2, "find: cannot stat %s\n", buf);
      continue;
    }

    if(strcmp(name, target) == 0){
      if(show_ino)
        printf("%s %d\n", buf, st.ino);
      else
        printf("%s\n", buf);
    }

    if(st.type == T_DIR){
      find(buf, target);
    }
  }

  close(fd);
}
```

Figure 4.1 – The search function of user/find.c

```
dev1@dev1:~/xv6-riscv$ sed -n '76,100p' user/find.c

int
main(int argc, char *argv[])
{
  if(argc < 3){
    fprintf(2, "usage: find <dir> <name> [-n]\n");
    exit(1);
  }

  if(argc >= 4 && strcmp(argv[3], "-n") == 0){
    show_ino = 1;
  }

  find(argv[1], argv[2]);
  exit(0);
}
```

Figure 4.2 – Argument handling in main()

4.2 Testing find

In the tree `/r2`, `/a/r2` and `/a/b/r2`, `find . r2` finds all three, and `find a b` finds the directory `a/b` (Figure 4.2). With `-n`, the three files named `r2` have three different inodes (32, 33, 2): the same name does not mean the same file. In the opposite case, after `ln cat c2`, `find . c2 -n` reports inode 3, and `mystat cat c2` confirms that `cat` and `c2` are one inode with 2 links. A missing argument prints the usage line, and a missing directory prints `find: cannot open nosuch`.

```
$ mkdir a/b
$ echo x > a/b/r2
$ echo y > a/r2
$ find . r2
./a/b/r2
./a/r2
./r2
$ find a b
a/b
$ find . r2 -n
./a/b/r2 32
./a/r2 33
./r2 2
$ ln cat c2
$ find . c2 -n
./c2 3
$ mystat cat c2
cat: file, inode 3, links 2, size 35344 bytes
c2: file, inode 3, links 2, size 35344 bytes
$ find
usage: find <dir> <name> [-n]
$ find nosuch r2
find: cannot open nosuch
```

Figure 4.3 – find results, inode numbers with -n, and error handling

CONCLUSIONS

The xv6 file system made the Lab 5 rules visible in the source. A directory is literally an array of 16-byte `(inum, name)` records, and `ls`, `mystat` and `find` only read those records and call `stat()`. The life of a file is controlled by two counters. `nlink` counts names on disk and is decremented by `sys_unlink()`; `ref` counts in-memory users. Blocks are freed by `itrunc()` in `iput()` only when both reach zero, which is why `r2` kept the full 2441 bytes after `README` was removed.

The limits follow directly from a few constants: 1 KiB blocks, 12 direct pointers plus one indirect block give a maximum file of 268 KiB, `DIRSIZ` = 14 silently truncates longer names, and the 2000-block disk spends 47 blocks on boot, superblock, log, inodes and bitmap. The comparison with Lab 5 shows what a production file system adds on top of the same design: owners and permissions, timestamps, symbolic links, larger and multi-level indirect blocks (extents in ext4), and errors instead of silent truncation. The `find` program needed no kernel support at all: recursion over `open`, `read` and `stat` was enough, as long as it skips `.` and `..` and closes every descriptor.

REFERENCES

1. Cox, R., Kaashoek, M. F., & Morris, R. *xv6: a simple, Unix-like teaching operating system* (RISC-V edition). MIT, 2024. Chapter 8: File system.

2. `mit-pdos/xv6-riscv`: `kernel/fs.h`, `kernel/fs.c`, `kernel/sysfile.c`, `mkfs/mkfs.c`. https://github.com/mit-pdos/xv6-riscv

3. Kerrisk, M. (2010). *The Linux Programming Interface*. No Starch Press. Ch. 14 and 18.